05 HAFTA

C# ile Nesneye Yönelik Programlama · Ders Notu

Tasarım Kalıplarına Giriş (GoF Patterns)

(Kanıtlanmış Çözümler I)

Bu Haftanın Kazanımları

Bu dersin sonunda öğrenciler:

- ✓ **design pattern (tasarım kalıbı)** kavramını, GoF kataloğunu ve kalıpların üç ana kategorisini açıklar.
- ✓ **Singleton** kalıbını thread-safe biçimde uygular ve Dependency Injection ile ilişkisini değerlendirir.
- ✓ **Factory Method** ile nesne oluşturma sorumluluğunu alt sınıflara devreder.
- ✓ **Adapter** ve **Decorator** kalıplarıyla uyumsuz arabirimleri bağlar, nesnelere dinamik davranış ekler.
- ✓ **Observer** ve **Strategy** kalıplarını C#'ın `event`, `delegate` ve `interface` olanaklarıyla uygular.

İçindekiler

1. Tasarım Kalıbı Nedir? Neden Kullanılır?
2. Creational Patterns: Singleton, Factory Method
3. Structural Patterns: Adapter, Decorator
4. Behavioral Patterns: Observer, Strategy

— Haftanın Özeti

— Anahtar Kavramlar

— Alıştırmalar

— Ek Okuma Kaynakları

1. Tasarım Kalıbı Nedir? Neden Kullanılır?

design pattern (tasarım kalıbı), yazılım tasarımında tekrar tekrar karşılaşılan bir probleme verilmiş, denenmiş ve isimlendirilmiş genel bir çözümdür. Kalıp kopyalanıp yapıştırılacak hazır bir kod değildir; belirli bir bağlamda nasıl bir sınıf ve nesne düzeni kurulacağını anlatan bir şablondur. Aynı kalıp farklı dillerde ve projelerde farklı kodlarla uygulanabilir.

Kavram, mimar Christopher Alexander'ın binalar ve şehirler için geliştirdiği kalıp dilinden yazılıma taşınmıştır. 1994 yılında Erich Gamma, Richard Helm, Ralph Johnson ve John Vlissides'in yazdığı *Design Patterns: Elements of Reusable Object-Oriented Software* kitabı 23 kalıbı katalog hâline getirmiştir. Yazarlar bugün kısaca **GoF (Gang of Four — Dörtlü Çete)** olarak anılır.

> **TANIM** **Bir kalıbın dört temel öğesi**
>
> - **Ad:** Ekipte ortak bir kelime dağarcığı oluşturur ("Burada bir Decorator kullanalım").
> - **Problem:** Kalıbın hangi durumda uygulanacağını tanımlar.
> - **Çözüm:** Sınıfları, sorumluluklarını ve aralarındaki ilişkileri anlatır.
> - **Sonuçlar:** Kalıbın getirdiği avantajları ve maliyetleri (esneklik, karmaşıklık, performans) ortaya koyar.

1.1 Neden kullanılır?

- **Ortak dil:** "Buraya bir Strategy koyalım" cümlesi, bir paragraflık açıklamanın yerini tutar.
- **Kanıtlanmış çözüm:** Binlerce projede sınanmış yapılar, aynı hataları tekrar yapmanızı önler.
- **Esneklik ve bakım kolaylığı:** Kalıpların çoğu, değişen parçayı değişmeyen parçadan ayırarak yeni gereksinimlerin mevcut kodu bozmadan eklenmesini sağlar.
- **OOP ilkeleriyle uyum:** Kalıplar büyük ölçüde encapsulation, polymorphism ve abstraction üzerine kuruludur (3. hafta).

> **BİLGİ** **GoF'un iki temel ilkesi**
>
> - Bir uygulamaya (somut sınıfa) değil, bir **interface (arabirim)**'e göre programlayın.
> - **Inheritance (kalıtım)** yerine **composition (bileşim)** tercih edin: davranışı alt sınıf türeterek değil, nesneleri bir araya getirerek oluşturun.

1.2 Kalıp kategorileri

Kategori	Amaç	GoF kalıpları
Creational (oluşturucu)	Nesne oluşturma sürecini soyutlar	**Singleton**, **Factory Method**, Abstract Factory, Builder, Prototype
Structural (yapısal)	Sınıf ve nesnelerin daha büyük yapılar hâlinde nasıl birleşeceğini düzenler	**Adapter**, **Decorator**, Bridge, Composite, Facade, Flyweight, Proxy
Behavioral (davranışsal)	Nesneler arası iletişimi ve sorumluluk dağılımını düzenler	**Observer**, **Strategy**, Chain of Responsibility, Command, Iterator, Mediator, Memento, State, Template Method, Visitor, Interpreter

> **DİKKAT** **Kalıp hastalığına dikkat**
>
> Kalıplar bir amaç değil, araçtır. Basit bir probleme üç katmanlı bir kalıp yapısı kurmak **over-engineering (aşırı mühendislik)** olur. Önce sorunu yaşayın, sonra onu çözen kalıbı uygulayın. Bu yaklaşım 7. haftada göreceğimiz **YAGNI** ilkesiyle de örtüşür.

2. Creational Patterns: Singleton, Factory Method

2.1 Singleton

Singleton, bir sınıfın uygulama boyunca yalnızca tek bir örneğinin olmasını garanti eder ve bu örneğe global bir erişim noktası sağlar. Uygulama ayarları, bir donanım kaynağına erişim veya pahalı bir bağlantı nesnesi gibi "tek olması gereken" nesneler için kullanılır.

İlk akla gelen uygulama aşağıdakidir; ancak çok thread'li ortamda hatalıdır:

C# · AyarlarHatali.cs

```
public class AyarlarHatali
{
    private static AyarlarHatali? _ornek;
    private AyarlarHatali() { }              // dışarıdan 'new' engellenir

    public static AyarlarHatali Ornek
    {
        get
        {
            if (_ornek == null)              // iki thread aynı anda girerse?
                _ornek = new AyarlarHatali(); // iki ayrı nesne oluşabilir!
            return _ornek;
        }
    }
}
```

İki thread aynı anda `if` kontrolüne girerse ikisi de nesnenin henüz oluşmadığını görür ve iki ayrı örnek yaratılır. Bu duruma **race condition (yarış durumu)** denir. .NET'te en temiz ve güvenli çözüm `Lazy<T>` kullanmaktır; `Lazy<T>` varsayılan olarak **thread-safe (iş parçacığı güvenli)** çalışır ve nesneyi ilk erişimde oluşturur (**lazy initialization — tembel başlatma**).

C# · UygulamaAyarlari.cs

```
public sealed class UygulamaAyarlari
{
    private static readonly Lazy<UygulamaAyarlari> _ornek =
        new(() => new UygulamaAyarlari());        // thread-safe ve tembel

    public static UygulamaAyarlari Ornek => _ornek.Value;

    public string UygulamaAdi { get; }
    public int SayfaBoyutu { get; }

    private UygulamaAyarlari()                    // private constructor
    {
        UygulamaAdi = "Okul Otomasyonu";
        SayfaBoyutu = 20;
        Console.WriteLine("Ayarlar yüklendi (yalnızca bir kez).");
    }
}
```

C# · Program.cs

```
var a1 = UygulamaAyarlari.Ornek;       // "Ayarlar yüklendi" burada yazılır
var a2 = UygulamaAyarlari.Ornek;       // aynı nesne döner
Console.WriteLine(ReferenceEquals(a1, a2));   // True
```

DİKKAT **Singleton'ın gölge yüzü**

Klasik Singleton global bir durum (state) oluşturur, bağımlılıkları gizler ve birim testte sahte (mock) bir nesneyle değiştirilmesi zordur. Ayrıca sınıf hem kendi işini hem de kendi yaşam döngüsünü yönettiği için tek sorumluluk ilkesini (6. hafta) zorlar.

İPUCU **Modern alternatif: DI container ile tekil yaşam süresi**

ASP.NET Core'da sınıfı sıradan bir sınıf olarak yazıp `builder.Services.AddSingleton<IAyarlar, Ayarlar>();` ile kaydetmek aynı "tek örnek" garantisini sağlar. Sınıf test edilebilir kalır, tekillik sorumluluğu container'a geçer. Bu konu 11. haftada işlenecektir.

2.2 Factory Method

Kodun her yerinde `new EpostaBildirim()` gibi somut sınıf oluşturmak, istemci kodu o sınıfa sıkı sıkıya bağlar. Yeni bir bildirim türü (SMS, push) eklendiğinde, nesne oluşturulan her yerin değiştirilmesi gerekir. **Factory Method (fabrika metodu)** kalıbı nesne oluşturmak için bir arabirim tanımlar, fakat hangi somut sınıfın oluşturulacağına alt sınıfların karar vermesini sağlar.

Rol	Örnekteki karşılığı
Product (ürün)	`IBildirim` arabirimi
Concrete Product	`EpostaBildirim`, `SmsBildirim`
Creator (yaratıcı)	`BildirimServisi`: iş mantığını içerir, `BildirimOlustur()` fabrika metodunu çağırır
Concrete Creator	`EpostaServisi`, `SmsServisi`: fabrika metodunu override eder

C# · Bildirimler.cs

```
// Product
public interface IBildirim
{
    void Gonder(string alici, string mesaj);
}

// Concrete products
public class EpostaBildirim : IBildirim
{
    public void Gonder(string alici, string mesaj) =>
        Console.WriteLine($"[E-posta] {alici}: {mesaj}");
}

public class SmsBildirim : IBildirim
{
    public void Gonder(string alici, string mesaj) =>
        Console.WriteLine($"[SMS] {alici}: {mesaj[..Math.Min(mesaj.Length, 160)]}");
}
```

C# · BildirimServisleri.cs

```csharp
// Creator: NE yapılacağını bilir, HANGİ nesneyle yapılacağını alt sınıfa bırakır
public abstract class BildirimServisi
{
    protected abstract IBildirim BildirimOlustur();    // factory method

    public void DuyuruYap(string alici, string mesaj)
    {
        IBildirim bildirim = BildirimOlustur();        // somut sınıfı bilmiyor
        bildirim.Gonder(alici, mesaj.Trim());
    }
}

public class EpostaServisi : BildirimServisi
{
    protected override IBildirim BildirimOlustur() => new EpostaBildirim();
}

public class SmsServisi : BildirimServisi
{
    protected override IBildirim BildirimOlustur() => new SmsBildirim();
}
```

C# · Program.cs

```csharp
string tercih = "sms";
BildirimServisi servis = tercih == "sms" ? new SmsServisi() : new EpostaServisi();
servis.DuyuruYap("ogrenci@universite.edu.tr", "Vize sınavı 12 Kasım'da.");
// Yeni bir tür (PushServisi) eklemek için mevcut sınıflara dokunmak gerekmez.
```

BİLGİ **Simple Factory ile karıştırmayın**

Pratikte sık görülen `static IBildirim Olustur(string tur) => tur switch { ... }` biçimindeki metot **Simple Factory** olarak adlandırılır. GoF kataloğunda yer almaz ama kullanışlıdır. Dezavantajı, her yeni tür için `switch` bloğunun değiştirilmesi gerekmesidir. Factory Method ise yeni türleri yeni bir alt sınıf ekleyerek destekler; bu fark 6. haftadaki Open/Closed ilkesinin ta kendisidir.

3. Structural Patterns: Adapter, Decorator

3.1 Adapter

Adapter (adaptör) kalıbı, bir sınıfın arabirimini istemcinin beklediği başka bir arabirime dönüştürür. Yurt dışında kullandığınız priz dönüştürücüsünü düşünün: ne cihazınızı ne de duvardaki prizi değiştirirsiniz; aralarına bir çevirici koyarsınız.

Senaryo: Uygulamamız ödemeleri `IOdemeIslemcisi` arabirimi üzerinden alıyor. Anlaşma yaptığımız bankanın kütüphanesi ise tutarı kuruş cinsinden `long` olarak istiyor ve başarı bilgisini bir durum koduyla döndürüyor. Kütüphanenin kaynak koduna erişimimiz yok.

C# · Odeme.cs

```csharp
// Target: uygulamamızın beklediği arabirim
public interface IOdemeIslemcisi
{
    bool Tahsil(decimal tutar, string kartNo);
}
```

C# · Odeme.cs

```csharp
// Adaptee: değiştiremediğimiz üçüncü parti kütüphane
public class EskiBankaApi
{
    // Tutarı kuruş cinsinden alır; başarılıysa 0 döndürür
    public int OdemeYap(long kurus, string kart, string paraBirimi)
    {
        Console.WriteLine($"Banka: {kurus} kuruş {paraBirimi} (...{kart[^4..]})");
        return 0;
    }
}

// Adapter: Target'ı uygular, çağrıyı Adaptee'nin diline çevirir
public class EskiBankaAdapter : IOdemeIslemcisi
{
    private readonly EskiBankaApi _banka;
    public EskiBankaAdapter(EskiBankaApi banka) => _banka = banka;

    public bool Tahsil(decimal tutar, string kartNo)
    {
        long kurus = (long)Math.Round(tutar * 100);
        return _banka.OdemeYap(kurus, kartNo, "TRY") == 0;
    }
}
```

C# · Program.cs

```csharp
IOdemeIslemcisi odeme = new EskiBankaAdapter(new EskiBankaApi());
bool basarili = odeme.Tahsil(249.90m, "4543123412341234");
// Banka değişirse yalnızca yeni bir adapter yazılır; istemci kod aynı kalır.
```

Bu örnekteki yapı **object adapter (nesne adaptörü)** olarak adlandırılır ve composition kullanır. C# sınıflarda çoklu kalıtımı desteklemediği için GoF'taki "class adapter" türü pratikte pek kullanılmaz.

3.2 Decorator

Decorator (dekoratör) kalıbı, bir nesneye çalışma zamanında, alt sınıf türetmeden yeni sorumluluklar ekler. Decorator, sarmaladığı nesneyle aynı arabirimi uygular; kendi işini yapar ve çağrıyı içerideki nesneye iletir. Böylece decorator'lar üst üste giydirilebilir.

Bir kahve dükkânı düşünelim: süt, karamel ve ekstra shot eklenebilir. Her kombinasyon için alt sınıf yazarsak (`SutluKahve`, `SutluKaramelliKahve`...) sınıf sayısı katlanarak artar; buna **class explosion (sınıf patlaması)** denir. Decorator ile her eklenti tek bir sınıftır.

C# · Icecekler.cs

```csharp
public interface IIcecek
{
    string Aciklama { get; }
    decimal Fiyat { get; }
}

public class FiltreKahve : IIcecek
{
    public string Aciklama => "Filtre kahve";
    public decimal Fiyat => 60m;
}
```

C# · Icecekler.cs

```
// Temel decorator: aynı arabirimi uygular ve bir IIcecek sarmalar
public abstract class IcecekDecorator : IIcecek
{
    protected readonly IIcecek Icecek;
    protected IcecekDecorator(IIcecek icecek) => Icecek = icecek;

    public virtual string Aciklama => Icecek.Aciklama;
    public virtual decimal Fiyat => Icecek.Fiyat;
}

public class SutEkle : IcecekDecorator
{
    public SutEkle(IIcecek icecek) : base(icecek) { }
    public override string Aciklama => base.Aciklama + " + süt";
    public override decimal Fiyat => base.Fiyat + 10m;
}

public class KaramelEkle : IcecekDecorator
{
    public KaramelEkle(IIcecek icecek) : base(icecek) { }
    public override string Aciklama => base.Aciklama + " + karamel";
    public override decimal Fiyat => base.Fiyat + 15m;
}
```

C# · Program.cs

```
IIcecek siparis = new KaramelEkle(new SutEkle(new SutEkle(new FiltreKahve())));
Console.WriteLine($"{siparis.Aciklama}: {siparis.Fiyat} TL");
// Filtre kahve + süt + süt + karamel: 95 TL
```

.NET'in kendisi de bu kalıbı yoğun biçimde kullanır. `Stream` sınıfları klasik bir decorator zinciridir: her katman aynı `Stream` arabirimini sunar ve bir öncekine davranış ekler.

C# · Program.cs

```
using System.IO.Compression;

using var dosya = File.Create("yedek.gz");                      // temel bileşen
using var sikistirma = new GZipStream(dosya, CompressionLevel.Optimal);
using var tampon = new BufferedStream(sikistirma);              // + tamponlama
using var yazici = new StreamWriter(tampon);
yazici.WriteLine("Sıkıştırılmış ve tamponlanmış veri");
```

	Adapter	Decorator
Amaç	Uyumsuz bir arabirimi beklenen arabirime çevirmek	Arabirimi değiştirmeden davranış eklemek
Arabirim	Farklı arabirim → hedef arabirim	Sarmaladığıyla aynı arabirim
Katman sayısı	Genellikle tek katman	Üst üste birçok katman
.NET'teki örnek	`StreamReader` (bayt akışını metin okuyucuya çevirir)	`BufferedStream`, `GZipStream`, `CryptoStream`

4. Behavioral Patterns: Observer, Strategy

4.1 Observer

Observer (gözlemci) kalıbı nesneler arasında bire-çok bir bağımlılık tanımlar: bir nesnenin durumu değiştiğinde, ona bağlı tüm nesneler otomatik olarak haberdar edilir. Durumu değişen nesneye **subject/publisher (yayıncı)**, haberdar edilenlere **observer/subscriber (abone)** denir. Yayıncı, abonelerinin somut tiplerini bilmez; bu da iki tarafı birbirinden bağımsız hâle getirir.

C#'ta `event` ve `delegate` mekanizması, dilin içine yerleştirilmiş bir Observer uygulamasıdır. Aşağıdaki örnekte ürünün stoğu değiştiğinde farklı aboneler kendi işlerini yapar:

C# · Urun.cs

```
public class StokDegistiEventArgs : EventArgs
{
    public required string UrunAdi { get; init; }
    public int EskiStok { get; init; }
    public int YeniStok { get; init; }
}

public class Urun
{
    private int _stok;
    public string Ad { get; }

    public Urun(string ad, int stok) { Ad = ad; _stok = stok; }

    // Subject: abonelerin kaydolduğu olay
    public event EventHandler<StokDegistiEventArgs>? StokDegisti;

    public int Stok
    {
        get => _stok;
        set
        {
            if (value == _stok) return;
            int eski = _stok;
            _stok = value;
            // ?.Invoke: hiç abone yoksa event null'dır
            StokDegisti?.Invoke(this, new StokDegistiEventArgs
                { UrunAdi = Ad, EskiStok = eski, YeniStok = value });
        }
    }
}
```

C# · Program.cs

```csharp
var laptop = new Urun("Laptop", 10);

// Abone 1: lambda ile kritik stok uyarısı
laptop.StokDegisti += (sender, e) =>
{
    if (e.YeniStok < 3)
        Console.WriteLine($"UYARI: {e.UrunAdi} kritik seviyede ({e.YeniStok})");
};

// Abone 2: bir nesnenin metodu
var logcu = new StokLogcu();
laptop.StokDegisti += logcu.Kaydet;

laptop.Stok = 5;     // [LOG] Laptop: 10 -> 5
laptop.Stok = 2;     // UYARI + [LOG] Laptop: 5 -> 2
laptop.StokDegisti -= logcu.Kaydet;   // abonelikten çık

class StokLogcu
{
    public void Kaydet(object? sender, StokDegistiEventArgs e) =>
        Console.WriteLine($"[LOG] {e.UrunAdi}: {e.EskiStok} -> {e.YeniStok}");
}
```

DİKKAT **Unutulan abonelikler bellek sızıntısıdır**

Yayıncı, `event` üzerinden abonelerine referans tutar. Uzun ömürlü bir yayıncıya (ör. bir singleton servis) abone olan kısa ömürlü nesneler `-=` ile abonelikten çıkmazsa GC onları toplayamaz. Bu, .NET uygulamalarındaki en yaygın **memory leak (bellek sızıntısı)** nedenlerinden biridir.

BİLGİ **IObservable ve Reactive Extensions**

.NET, kalıbın arabirim tabanlı bir sürümünü `IObservable<T>` ve `IObserver<T>` ile de sunar. Sürekli akan veriler (sensör verileri, borsa fiyatları) üzerinde filtreleme ve birleştirme yapmak için bu arabirimler üzerine kurulu **Reactive Extensions** (System.Reactive) kütüphanesi kullanılabilir.

4.2 Strategy

Strategy (strateji) kalıbı, bir algoritma ailesini tanımlar, her birini ayrı bir sınıfa kapsüller ve bunların çalışma zamanında birbirinin yerine kullanılabilmesini sağlar. Bir metodun içinde büyüyen `if/else` veya `switch` blokları, genellikle bir Strategy kalıbına ihtiyaç olduğunun işaretidir.

Senaryo: Bir e-ticaret sitesinde kargo ücreti, seçilen kargo türüne göre farklı hesaplanıyor. Yeni kargo türlerinin sipariş hesaplama kodunu değiştirmeden eklenebilmesini istiyoruz.

C# · Kargo.cs

```csharp
public interface IKargoStratejisi
{
    string Ad { get; }
    decimal UcretHesapla(decimal tutar, double kg);
}
```

C# · Kargo.cs

```csharp
public class StandartKargo : IKargoStratejisi
{
    public string Ad => "Standart (3-5 gün)";
    public decimal UcretHesapla(decimal tutar, double kg) => 40m + (decimal)kg * 5m;
}

public class HizliKargo : IKargoStratejisi
{
    public string Ad => "Hızlı (ertesi gün)";
    public decimal UcretHesapla(decimal tutar, double kg) => 90m + (decimal)kg * 8m;
}

public class UcretsizKargo : IKargoStratejisi
{
    public string Ad => "Ücretsiz (1000 TL üzeri)";
    public decimal UcretHesapla(decimal tutar, double kg) => 0m;
}

// Context: stratejiyi kullanır ama hangisi olduğunu bilmez
public class SiparisHesaplayici
{
    private IKargoStratejisi _strateji;
    public SiparisHesaplayici(IKargoStratejisi strateji) => _strateji = strateji;
    public void StratejiDegistir(IKargoStratejisi yeni) => _strateji = yeni;

    public decimal ToplamHesapla(decimal sepet, double kg) =>
        sepet + _strateji.UcretHesapla(sepet, kg);
}
```

C# · Program.cs

```csharp
var hesap = new SiparisHesaplayici(new StandartKargo());
Console.WriteLine(hesap.ToplamHesapla(500m, 2));   // 550

hesap.StratejiDegistir(new HizliKargo());
Console.WriteLine(hesap.ToplamHesapla(500m, 2));   // 606

// Stratejiyi çalışma zamanında koşullara göre seçmek
IKargoStratejisi Sec(decimal tutar, bool acil) => (tutar, acil) switch
{
    (_, true)        => new HizliKargo(),
    (> 1000m, false) => new UcretsizKargo(),
    _                => new StandartKargo()
};
Console.WriteLine(Sec(1500m, false).Ad);           // Ücretsiz (1000 TL üzeri)
```

Tek bir metottan oluşan basit stratejiler için ayrı sınıflar yazmak zorunda değilsiniz. C#'ta `Func<>` **delegate (temsilci)** tipleri hafif bir Strategy işlevi görür. Günlük hayatta her gün kullandığınız `OrderBy(o => o.Soyad)` çağrısındaki lambda da, sıralama algoritmasına verilen bir stratejidir.

C# · Program.cs

```csharp
Func<decimal, double, decimal> kampanya = (tutar, kg) => tutar > 500m ? 20m : 45m;
Console.WriteLine(kampanya(750m, 1.5));            // 20

var ogrenciler = new List<string> { "Zeynep", "Ali", "Mert" };
// Sort metoduna karşılaştırma stratejisi veriliyor
ogrenciler.Sort((a, b) => a.Length.CompareTo(b.Length));
```

Haftanın Özeti

- Tasarım kalıpları, tekrar eden problemlere verilmiş isimlendirilmiş çözümlerdir. GoF kataloğu 23 kalıbı creational, structural ve behavioral olarak üç gruba ayırır.
- Kalıplar ortak bir dil sağlar, fakat gereksiz yere uygulandığında karmaşıklığı artırır.
- Bu hafta işlenen altı kalıbın özeti:

Kalıp	Kategori	Çözdüğü problem	.NET'teki karşılığı
Singleton	Creational	Bir sınıftan tek örnek olması	`Lazy<T>`, DI'da `AddSingleton`
Factory Method	Creational	Somut sınıfa bağımlı nesne oluşturma	`ILoggerFactory`, `IHttpClientFactory`
Adapter	Structural	Uyumsuz arabirimleri birlikte çalıştırma	`StreamReader`, üçüncü parti SDK sarmalayıcıları
Decorator	Structural	Alt sınıf türetmeden davranış ekleme	`BufferedStream`, `GZipStream`
Observer	Behavioral	Durum değişikliğini çok sayıda nesneye duyurma	`event`, `IObservable<T>`
Strategy	Behavioral	Değiştirilebilir algoritma ailesi	`IComparer<T>`, LINQ'e verilen lambda'lar

Anahtar Kavramlar

Terim	Türkçe karşılığı	Kısa açıklama
design pattern	tasarım kalıbı	Tekrar eden bir tasarım problemine isimlendirilmiş genel çözüm
creational pattern	oluşturucu kalıp	Nesne oluşturma sürecini soyutlayan kalıp
structural pattern	yapısal kalıp	Sınıf ve nesnelerin bir araya getirilişini düzenleyen kalıp
behavioral pattern	davranışsal kalıp	Nesneler arası iletişimi düzenleyen kalıp
composition	bileşim	Davranışı nesneleri bir araya getirerek oluşturma
thread-safe	iş parçacığı güvenli	Birden fazla thread aynı anda kullandığında doğru çalışan kod
race condition	yarış durumu	Sonucun thread'lerin çalışma sırasına bağlı olduğu hata
lazy initialization	tembel başlatma	Nesnenin ilk ihtiyaç duyulduğu anda oluşturulması
class explosion	sınıf patlaması	Her kombinasyon için alt sınıf yazılmasıyla sınıf sayısının katlanması
publisher / subscriber	yayıncı / abone	Observer kalıbında olayı yayınlayan ve dinleyen taraflar
memory leak	bellek sızıntısı	Artık gerekmeyen nesnelerin referans tutulduğu için toplanamaması
delegate	temsilci	Bir metoda referans tutan tip (`Func<>`, `Action<>`)

Alıştırmalar

1. `Lazy<T>` kullanarak thread-safe bir `Logger` singleton'ı yazın. `Parallel.For(0, 100, ...)` ile 100 thread'den erişip constructor'ın yalnızca bir kez çalıştığını gösterin. Ardından aynı testi hatalı (kilitsiz) sürümle tekrarlayıp sonuçları karşılaştırın.
2. Factory Method ile bir rapor dışa aktarma altyapısı kurun: `RaporDisaAktarici` (creator), `CsvDisaAktarici` ve `JsonDisaAktarici` (concrete creator), `IRaporFormati` (product).
3. Sıcaklığı Fahrenheit olarak `"72.5F"` biçiminde bir string döndüren hayali bir `EskiHavaDurumuServisi` yazın. Uygulamanızın beklediği `IHavaDurumu.SicaklikCelsius(string sehir)` arabirimine uyan bir adapter geliştirin.
4. `IMetinIsleyici` arabirimi için `BuyukHarfDecorator`, `KirpmaDecorator` ve `SansurDecorator` sınıflarını yazın. Decorator'ları farklı sıralarla giydirerek sıranın sonucu nasıl etkilediğini açıklayın.
5. Bir `HavaIstasyonu` sınıfı, sıcaklık değiştiğinde `event` yayınlasın. Ekran, alarm ve istatistik olmak üzere üç abone yazın; alarm abonesi 40 derece üzerinde uyarı versin ve 3 uyarıdan sonra abonelikten çıksın.
6. Öğrenci, kampanya ve sadakat indirimlerini Strategy kalıbıyla modelleyin. Aynı çözümü `Func<decimal, decimal>` ile yeniden yazın; hangi durumda hangisini tercih edeceğinizi gerekçelendirin.
7. .NET'te `IEnumerable<T>/IEnumerator<T>`, `event`, `Stream` ve `IComparer<T>` yapılarının hangi GoF kalıplarına karşılık geldiğini birer paragrafla açıklayın.

Ek Okuma Kaynakları

- Refactoring.Guru — Tasarım kalıpları kataloğu (C# örnekleriyle)
- Observer design pattern — .NET belgeleri
- Events — C# programlama kılavuzu
- Lazy<T> sınıfı — API başvurusu
- Jon Skeet — Implementing the Singleton Pattern in C#
- Kitap: E. Gamma, R. Helm, R. Johnson, J. Vlissides — *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.